Insignia

Empowering *Insights*, Driving *A.I Innovation*

INTERNAL TRAINING

# Big Data

## Sesi 4 - Batch Processing Framework (Spark)

Memproses data skala besar secara terdistribusi dengan Apache Spark.

| HARI | SLOT SIANG | SLOT SORE | LOKASI |
| --- | --- | --- | --- |
| Rabu, 2 Desember 2026 | 13.30 - 15.30 WIB | 15.30 - 17.30 WIB | Kantor Insignia / online |

AGENDA

# Rencana Sesi 2 Jam

| | | | |
|---|---|---|---|
| **13.30 - 13.40** | Pembukaan dan recap sesi sebelumnya | PEMBUKAAN | 10 mnt |
| **13.40 - 14.05** | Konsep: driver, executor, partition | KONSEP | 25 mnt |
| **14.05 - 14.20** | Lazy evaluation dan DAG | KONSEP | 15 mnt |
| **14.20 - 15.00** | **Hands-on 1: agregasi batch di Spark** | HANDS-ON | 40 mnt |
| **15.00 - 15.20** | **Hands-on 2: partition dan join** | HANDS-ON | 20 mnt |
| **15.20 - 15.30** | Recap dan tanya jawab | PENUTUP | 10 mnt |

Slot sore (15.30 - 17.30) memakai susunan yang sama.

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

TUJUAN PEMBELAJARAN

# Setelah Sesi Ini, Kamu Mampu

**01**

## Menjelaskan

peran driver, executor, dan partition saat Spark memproses data.

**02**

## Menjalankan

transformasi batch di DataFrame Spark dari file CSV.

**03**

## Membaca

DAG dan stage di Spark UI untuk menemukan shuffle.

**04**

## Mengoptimasi

jumlah partition dan strategi join sederhana.

**Target akhir sesi:** setiap peserta menghasilkan ringkasan revenue per kota dan kategori dari 1 juta baris data.

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

Insignia
Empowering *Insights*, Driving *A.I Innovation*

01

# Konsep Inti Spark

Kenapa satu mesin tidak cukup, dan bagaimana Spark membagi pekerjaan.

DI BAGIAN INI


01 Batas satu mesin

02 Driver, executor, partition

03 Lazy evaluation dan DAG

04 Narrow vs wide transformation

MASALAH

# Kenapa Pandas Mentok di Data Besar

1. **Batas memori satu mesin**
   Seluruh DataFrame harus muat di RAM. Data melebihi RAM berarti proses gagal.

2. **Hanya satu core bekerja**
   Sebagian besar operasi Pandas berjalan di satu thread, core lain menganggur.

3. **Tidak tahan kegagalan**
   Jika proses mati di tengah jalan, pekerjaan diulang dari awal.

> **Masalahnya bukan di kode, tapi di batas satu mesin.**

**Spark menjawab ketiganya dengan membagi data ke banyak executor dan mengulang task yang gagal.**

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

SKALA DATA

# Data yang Dipakai di Hands-on

**1 juta**

**baris transaksi**

Dataset sintetis
retail_transactions.csv

**93,5 MB**

**ukuran file CSV**

Cukup untuk melihat perbedaan
Pandas dan Spark

**200**

**shuffle partitions default**

Nilai bawaan
spark.sql.shuffle.partitions

**Kesimpulan:** data ini masih muat di laptop, jadi perbandingan Pandas dan Spark tetap adil.

Sumber: generator dataset sintetis modul Big Data (seed 42) dan dokumentasi konfigurasi Spark SQL.

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

PERBANDINGAN

# Pandas vs Spark

| Aspek | Pandas | Spark |
|---|---|---|
| **Model eksekusi** | Eager, langsung dijalankan | Lazy, dijalankan saat ada action |
| **Skala** | Terbatas RAM satu mesin | Total memori seluruh cluster |
| **Paralelisme** | Umumnya satu core | Banyak task di banyak executor |
| **Saat gagal** | Ulang dari awal | Task yang gagal diulang otomatis |
| **Cocok untuk** | Eksplorasi data kecil-menengah | Data besar dan pipeline produksi |

**Spark tidak selalu lebih cepat. Untuk data kecil, overhead distribusinya membuat Pandas sering lebih cepat.**

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

# Narrow vs Wide Transformation

NARROW

## Tanpa shuffle

Setiap partition output hanya butuh satu partition input. Bisa dijalankan dalam satu stage.

CONTOH

- `filter()`
- `select()`
- `withColumn()`

WIDE

## Butuh shuffle

Data dari banyak partition harus dipindah antar executor. Memecah job menjadi stage baru.

CONTOH

- `groupBy().agg()`
- `join()`
- `orderBy()`

**Shuffle adalah operasi paling mahal di Spark. Kurangi wide transformation yang tidak perlu.**

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

ALUR EKSEKUSI

# Dari Kode ke Hasil di Spark

1

**Action dipanggil**

show(), count(), atau write() memicu eksekusi.

2

**DAG dibentuk**

Driver menyusun rencana dari semua transformasi.

3

**Dibagi jadi stage**

Setiap shuffle memisahkan stage baru.

4

**Task per partition**

Executor menjalankan satu task per partition.

5

**Hasil dikumpulkan**

Output ditulis ke storage atau dikirim ke driver.

**Transformasi tanpa action tidak menjalankan apa pun. Inilah lazy evaluation.**

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

ISTILAH KUNCI

# Lima Istilah yang Muncul di Spark UI

| | | |
|---|---|---|
| 01 | **Driver** | Proses utama yang menyusun rencana eksekusi dan membagi task. |
| 02 | **Executor** | Proses di worker yang menjalankan task dan menyimpan data sementara. |
| 03 | **Partition** | Potongan data. Satu partition diproses oleh satu task. |
| 04 | **Stage** | Kumpulan task yang bisa jalan tanpa memindahkan data antar executor. |
| 05 | **Shuffle** | Pemindahan data antar executor. Paling mahal dari sisi waktu dan jaringan. |

Buka Spark UI di tab Jobs dan Stages untuk melihat kelima istilah ini secara langsung.

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

# Agregasi Batch dengan DataFrame

*PySpark*

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("bigdata_s04").getOrCreate()

df = spark.read.csv("retail_transactions.csv", header=True, inferSchema=True)

summary = (
    df.withColumn("revenue", F.col("quantity") * F.col("unit_price"))
      .groupBy("city", "category")
      .agg(F.sum("revenue").alias("total_revenue"),
           F.count("transaction_id").alias("n_trx"))
      .orderBy(F.desc("total_revenue"))
)

summary.show(10)
```

*Semua transformasi di atas baru dieksekusi saat action show() dipanggil.*

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

# Langkah dan Checkpoint

1. **Load data**
   Baca retail_transactions.csv ke DataFrame Spark.

2. **Tambah kolom**
   Hitung revenue = quantity x unit_price.

3. **Agregasi**
   Kelompokkan per city dan category.

4. **Simpan hasil**
   Tulis summary ke format Parquet.

CHECKPOINT

## Expected output

- 48 baris (8 kota x 6 kategori)
- Kolom: city, category, total_revenue, n_trx
- Jumlah n_trx = 1.000.000

Tertinggal? buka notebook SOLUTION dan lanjut dari langkah terakhir yang berhasil.

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

CHECKLIST

# Sebelum Sesi Selesai

- [ ] Spark session berjalan tanpa error
- [ ] Data berhasil di-load (1.000.000 baris)
- [ ] Kolom revenue sudah terbentuk
- [ ] Summary berisi 48 baris
- [ ] Hasil tersimpan sebagai Parquet
- [ ] Stage dengan shuffle ditemukan di Spark UI

Detail langkah dan expected output ada di Modul Praktikum.

©2026 INSIGNIA. All contents and materials in this presentation belong to INSIGNIA

RINGKASAN

# Ringkasan Sesi 4

- Spark membagi data ke banyak executor lewat partition
- Transformasi bersifat lazy dan baru jalan saat ada action
- Wide transformation memicu shuffle, operasi paling mahal

**Sesi berikutnya:** Sesi 5 - Real-Time Processing Framework (Kafka/Flink), Kamis 3 Desember 2026

Insignia

Empowering *Insights*, Driving *A.I Innovation*

Q & A

# Terima Kasih

## Pertanyaan dan diskusi

**Sesi berikutnya:** Sesi 5 - Real-Time Processing Framework (Kafka/Flink)

Kamis, 3 Desember 2026

**Kontak: Fahmi, Mentor Data & AI**